UNIVERSIDAD INDUSTRIAL DE SANTANDER
Escuela de Ingeniería de Sistemas e Informática

Informe final del proyecto

Plataforma de Simulación de Inversión

Asignatura: Ingeniería de Software I · Docente: Gabriel Pedraza Ferreira

Integrantes
Juan Daniel Torres Ramírez
Camilo Andrés Velandia Mendoza
Santiago Fonseca Hernández

Bucaramanga, 3 de noviembre de 2026
Repositorio: github.com/Judator07/proyecto-clase-software1
Aplicación: simulador-inversion.juandanieltorresr.workers.dev

Contenido

1. Problema
- 1.1 Descripción
- 1.2 Caracterización
- 1.3 Necesidades identificadas
- 1.4 Solución propuesta

2. Requerimientos del sistema
- 2.1 Niveles de requerimientos
- 2.2 Puntos por definir resueltos
- 2.3 Cambios de la versión 1.1 del SRS
- 2.4 Interfaces de usuario

3. Análisis del sistema (UML)
- 3.1 Diagrama de clases
- 3.2 Diagrama de casos de uso
- 3.3 Diagrama de estados de la Transacción
- 3.4 Diagrama de estados de la cuenta del Inversionista
- 3.5 Diagrama de actividades de la orden de compra
- 3.6 Trazabilidad con el SRS

4. Diseño del sistema
- 4.1 Arquitectura general y despliegue
- 4.2 Arquitectura en capas (RNF-11)
- 4.3 Modelo de datos
- 4.4 Comportamiento: secuencia de la compra
- 4.5 API REST
- 4.6 Tabla de pantallas
- 4.7 Decisiones de diseño
- 4.8 Seguridad

5. Validación del sistema
- 5.1 Estrategia de pruebas
- 5.2 Cobertura
- 5.3 Matriz de trazabilidad: requerimiento → prueba → resultado
- 5.4 Prueba de carga (RNF-01, RNF-02)
- 5.5 Defectos encontrados por las pruebas y las revisiones
- 5.6 Pruebas pendientes a cargo del equipo

6. Descripción del proceso
- 6.1 Modelo de proceso
- 6.2 Sprints
- 6.3 Roles y distribución del trabajo (propuesta)
- 6.4 Uso de inteligencia artificial

7. Conclusiones
- 7.1 Síntesis de resultados
- 7.2 Análisis
- 7.3 Riesgos y limitaciones

8. Perspectivas
- 8.1 Validación pendiente (antes de la entrega final)
- 8.2 Producto
- 8.3 Técnica
- 8.4 Proceso

1. Problema

1.1 Descripción

Aprender a invertir suele costar dinero. Una persona sin formación financiera que quiere entender cómo funcionan las acciones, los fondos cotizados (ETF), las criptomonedas o el interés compuesto tiene pocas alternativas:

- **Teoría sin práctica.** Libros, cursos o videos explican los conceptos, pero no dejan experimentar sus consecuencias.
- **Práctica con dinero real.** Abrir una cuenta en un corredor de bolsa o en una plataforma de criptomonedas expone a pérdidas reales justo en la etapa en que más errores se cometen.

En la práctica, conceptos como **riesgo**, **volatilidad**, **diversificación**, **costo promedio** o **rendimiento** solo se interiorizan cuando se ven moverse con datos reales a lo largo del tiempo. Sin un entorno seguro, el aprendizaje queda incompleto o resulta costoso.

1.2 Caracterización

Aspecto	Caracterización
Afectados	Estudiantes universitarios y personas interesadas en aprender a invertir, con conocimientos financieros básicos o nulos (SRS §2.3).
Síntomas	Miedo a perder dinero, decisiones impulsivas sin entender el riesgo, desconocimiento del efecto del tiempo (interés compuesto) y de la diversificación.
Causas	No hay una herramienta gratuita, en español, que combine datos de mercado reales con dinero ficticio, retroalimentación sobre el rendimiento y contenido educativo.
Consecuencias	Aprendizaje lento o costoso. Pérdidas reales evitables y exposición a fraudes, sobre todo con criptomonedas.
Restricciones del contexto	Proyecto académico de 16 semanas con un equipo de 3 estudiantes. Proveedor de datos gratuito con límites de consultas. No se maneja dinero real (RNF-03). Debe cumplirse la Ley 1581 de 2012 de protección de datos (RNF-07).

1.3 Necesidades identificadas

A partir de los objetivos de la propuesta del proyecto, el SRS v1.0 identifica cuatro necesidades:

ID	Necesidad
N-01	Contar con una herramienta educativa para comprender el funcionamiento de las inversiones.
N-02	Crear y gestionar portafolios de inversión simulados, sin arriesgar dinero real.
N-03	Disponer de datos de mercado realistas y actualizados durante la simulación.
N-04	Recibir retroalimentación y análisis sobre el rendimiento de las inversiones.

1.4 Solución propuesta

Una **plataforma web educativa** en la que cada usuario recibe un saldo ficticio de **USD 10.000**. Con ese saldo puede:

- comprar y vender **acciones, ETF y criptomonedas** a precios reales y actualizados;

- simular **cuentas de ahorro con interés compuesto**;
- organizar sus inversiones en **varios portafolios**;
- analizar su rendimiento con **gráficos y estadísticas**;
- proyectar el efecto de **escenarios de mercado** (crisis, mercado alcista, alza de tasas);
- aprender en un **centro de contenido educativo**.

El equipo añadió dos ampliaciones a la propuesta original:

- las **criptomonedas** como cuarto tipo de instrumento;
- la **bancarrota simulada**: si el patrimonio cae por debajo de USD 100, el usuario puede reiniciar su cuenta. Un contador de bancarrotas sirve como retroalimentación.

Fuera del alcance: operaciones con dinero real, conexión con corredores, asesoría financiera personalizada y aplicaciones móviles nativas (SRS §1.4).

2. Requerimientos del sistema

La especificación completa está en el SRS v1.0 (plantilla IEEE 830 / Wiegers). Esta sección resume los requerimientos por nivel de abstracción y documenta los cambios de la **versión 1.1**, que surgieron del análisis UML y de la implementación.

2.1 Niveles de requerimientos

Nivel	Cantidad	Identificadores
Necesidades	4	N-01 a N-04
Requerimientos de usuario	6	RU-01 a RU-06
Requerimientos funcionales	22	RF-01 a RF-22
Requerimientos no funcionales	12	RNF-01 a RNF-12
Reglas de negocio	9	RN-01 a RN-09
Interfaces de usuario	11	IU-01 a IU-11

Requerimientos de usuario y trazabilidad

RU	Funcionalidad	Necesidades	RF
RU-01	Registro y perfil	N-01, N-02	RF-01 a RF-05
RU-02	Simulación de inversiones	N-01, N-02, N-03	RF-06 a RF-11
RU-03	Portafolios	N-02, N-04	RF-12 a RF-14
RU-04	Datos y gráficos	N-03, N-04	RF-15 a RF-18
RU-05	Aprendizaje y guías	N-01	RF-19, RF-20
RU-06	Simulación de escenarios	N-01, N-04	RF-21, RF-22

Requerimientos funcionales

ID	Nombre	Tipo	Prioridad	Crítico
RF-01	Registrar usuario	Necesario	Alta	Sí
RF-02	Iniciar y cerrar sesión	Necesario	Alta	Sí
RF-03	Consultar perfil de usuario	Necesario	Media	No
RF-04	Actualizar perfil de usuario	Deseable	Baja	No
RF-05	Reiniciar cuenta por bancarrota	Deseable	Media	No
RF-06	Consultar catálogo de instrumentos financieros	Necesario	Alta	Sí
RF-07	Consultar precios actuales e históricos	Necesario	Alta	Sí
RF-08	Comprar instrumento financiero	Necesario	Alta	Sí
RF-09	Vender instrumento o retirar ahorro	Necesario	Alta	Sí
RF-10	Simular cuenta de ahorro con interés compuesto	Necesario	Media	No
RF-11	Consultar historial de transacciones	Necesario	Media	No
RF-12	Crear portafolio	Necesario	Alta	Sí
RF-13	Gestionar portafolio (renombrar o eliminar)	Necesario	Media	No

ID	Nombre	Tipo	Prioridad	Crítico
RF-14	Consultar rendimiento y balance de portafolio	Necesario	Alta	Sí
RF-15	Generar gráficos de rendimiento	Necesario	Media	No
RF-16	Mostrar estadísticas y tendencias	Deseable	Baja	No
RF-17	Actualizar precios de mercado	Necesario	Alta	Sí
RF-18	Gestionar catálogo de instrumentos	Necesario	Media	No
RF-19	Consultar contenido educativo	Necesario	Media	No
RF-20	Gestionar contenido educativo	Necesario	Baja	No
RF-21	Simular escenario de mercado	Necesario	Media	No
RF-22	Ajustar variables del escenario	Necesario	Media	No

Requerimientos no funcionales

ID	Nombre	Atributo de calidad	Criterio de aceptación (resumen)
RNF-01	Tiempo de respuesta	Eficiencia	p95 ≤ 3 s con 50 usuarios virtuales
RNF-02	Usuarios concurrentes	Concurrencia	10 min con 50 usuarios: error < 1 % y saldos consistentes
RNF-03	Uso exclusivamente educativo	Seguridad (dominio)	Aviso en todas las pantallas y ningún dato de pago
RNF-04	Tolerancia a fallas del proveedor	Fiabilidad	Funciona con el último precio y muestra aviso de precio desactualizado
RNF-05	Autenticación y contraseñas	Seguridad	Contraseña de 8 o más caracteres con letra y número, hash con sal, HTTPS
RNF-06	Control de acceso y sesión	Seguridad	Rol administrador, acceso solo a lo propio, expiración a los 30 min
RNF-07	Protección de datos personales	Seguridad (legal)	Ley 1581: consentimiento obligatorio y política accesible en el pie de página
RNF-08	Facilidad de uso	Usabilidad	Registro y primera compra en ≤ 5 min; compra en ≤ 3 pasos
RNF-09	Disponibilidad	Disponibilidad	≥ 95 % durante el periodo de evaluación
RNF-10	Consistencia de operaciones	Integridad	Operaciones atómicas; saldo nunca negativo
RNF-11	Facilidad de mantenimiento	Mantenibilidad	Capas, control de versiones, pruebas de cálculos con cobertura ≥ 70 %
RNF-12	Acceso desde distintos navegadores	Accesibilidad	Chrome, Firefox y Edge; pantallas desde 360 px; contraste WCAG 2.1 AA

Reglas de negocio

ID	Regla
RN-01	Saldo inicial ficticio de USD 10.000,00. No hay depósitos ni retiros de dinero real.
RN-02	Acciones y ETF en unidades enteras mayores que cero; criptomonedas con hasta 8 decimales.

ID	Regla
RN-03	Las operaciones se ejecutan al último precio registrado, sin comisiones ni órdenes programadas.
RN-04	Sin ventas en corto ni apalancamiento.
RN-05	Máximo 10 portafolios por usuario, con nombres únicos.
RN-06	El costo de una posición es el costo promedio ponderado de sus compras.
RN-07	Al eliminar un portafolio, sus posiciones se liquidan al precio actual y el valor pasa al saldo.
RN-08	La simulación de escenarios es solo una proyección: no modifica datos.
RN-09	Bancarrota simulada si el patrimonio total es menor que USD 100,00; solo entonces se puede reiniciar la cuenta.

2.2 Puntos por definir resueltos

TBD	Decisión	Justificación
TBD-1 Proveedor de datos	**Yahoo Finance** (API de gráficos `v8/finance/chart`), sin API key	Una sola llamada entrega el precio actual y hasta 5 años de cierres diarios de acciones, ETF y criptomonedas (`BTC-USD`). Es una API no oficial: el riesgo se mitiga con históricos precargados y con el último precio guardado (RNF-04).
TBD-2 Tecnologías	TypeScript; React + Vite (cliente); Hono en **Cloudflare Workers** (servidor); **Cloudflare D1** (SQLite, relacional); despliegue con Workers Builds	Infraestructura gratuita, HTTPS por defecto, base de datos relacional administrada y un único lenguaje en todas las capas. Detalle en la sección de diseño.
TBD-3 Acciones y ETF	Acciones: AAPL, MSFT, GOOGL, AMZN, NVDA, TSLA, JPM, KO. ETF: SPY, QQQ, VTI, EEM, GLD	Alta liquidez, sectores variados y ETF de índice amplio, emergentes y oro, para practicar diversificación.
TBD-4 Criptomonedas	BTC, ETH, SOL, XRP, ADA	Las de mayor capitalización, con perfiles de volatilidad distintos.

2.3 Cambios de la versión 1.1 del SRS

El Taller de análisis UML registró cuatro hallazgos sobre el SRS v1.0 (Taller §8). Al revisarlos de nuevo, de forma crítica, contra el SRS durante la implementación se obtuvieron las siguientes decisiones:

#	Hallazgo o vacío	Decisión v1.1
1	Las transacciones no tenían un estado previo a su desenlace (hallazgo 1 del Taller).	Se acepta el estado **Pendiente** en el análisis. En el diseño corresponde al paso de confirmación (resumen de la orden) y **no se persiste**: la orden se registra al confirmarse como Ejecutada o Rechazada, o como Cancelada si el usuario desiste en el resumen. Así se evitan órdenes huérfanas y dobles ejecuciones.
2	No se definía si una cuenta en bancarrota puede comprar (hallazgo 2).	Se confirma el supuesto: **en bancarrota no se permiten compras ni depósitos**. Sí se permite vender, consultar y reiniciar.
3	El Taller afirma que las funciones del Administrador no tienen RF (hallazgo 3).	**El hallazgo era incorrecto**: el SRS v1.0 sí incluye RF-18 (Gestionar catálogo) y RF-20 (Gestionar contenido educativo). Se corrigieron las etiquetas del diagrama de casos de uso.
4	No se definía cómo se acreditan los intereses de la cuenta de ahorro (hallazgo 4).	RF-10 sí da la fórmula. Se precisa que el valor a la fecha es VF = P·(1 + r/n)^(n·t), con *t* igual al tiempo transcurrido desde el depósito y con tope en el plazo pactado. Se agrega la clase **DepósitoAhorro** al modelo.

#	Hallazgo o vacío	Decisión v1.1
5	En el diagrama de clases, `Transacción.tipo` solo admitía Compra y Venta.	Se amplía a **Compra, Venta, Depósito y Retiro**, como define el glosario del SRS.
6	El diagrama de casos de uso no incluía la simulación de la cuenta de ahorro (RF-10) ni la actualización de precios (RF-17).	Se agregan ambos casos de uso.
7	RF-16 no definía con precisión la volatilidad ni la tendencia.	**Volatilidad**: desviación estándar muestral (n − 1) de los rendimientos diarios del rango, en %. **Tendencia**: alcista si el último precio está por encima de la media móvil de 20 días; bajista en caso contrario.
8	RF-21 no decía cómo se aplica el horizonte.	Las variaciones de precio se aplican una sola vez. Los depósitos de ahorro se proyectan con la nueva tasa durante todo el horizonte.
9	RF-17 pide "guardar un registro de precio por consulta".	Se guarda el **precio actual** del instrumento y el **cierre diario** (el último precio del día). Los registros cada 15 minutos no los consume ningún requerimiento y multiplicarían el almacenamiento, por lo que no se conservan.
10	El intervalo de RF-17 es "ajustable a los límites del proveedor".	Variable de configuración `INTERVALO_ACTUALIZACION_MIN` (15 por defecto).
11	No se definía cuándo se evalúa la bancarrota (RN-09).	Se evalúa al consultar el perfil y el panel, y antes de cada compra o depósito.

2.4 Interfaces de usuario

Las 11 pantallas del SRS (IU-01 a IU-11) se implementaron. Se agregaron tres más:

- una página de inicio pública;
- la página de **política de tratamiento de datos** exigida por RNF-07;
- la página de detalle de un artículo educativo.

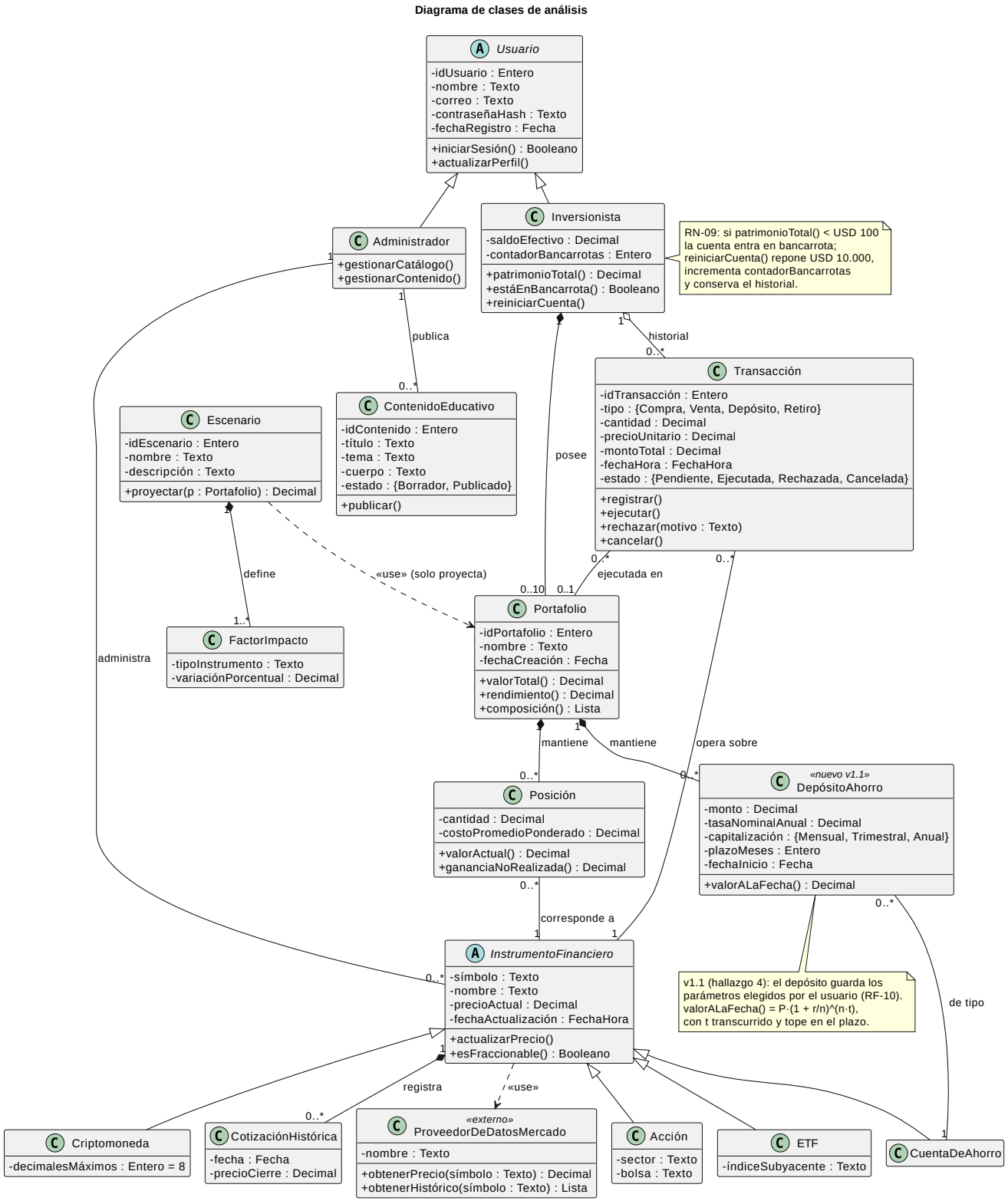
La correspondencia entre cada pantalla, su ruta y los RF está en la sección de diseño.

3. Análisis del sistema (UML)

El análisis orientado a objetos se elaboró en el Taller de análisis UML. Los diagramas de esta sección son la versión corregida según el SRS v1.1. Las fuentes PlantUML están en `docs/uml/analisis/` y se generaron con PlantUML 1.2025.4.

Diagrama	Tipo de modelo	Pregunta que responde
Clases	Estático (datos)	Qué conceptos existen y cómo se relacionan
Casos de uso	Comportamiento (vista externa)	Quién usa el sistema y qué puede hacer
Estados de la Transacción	Estado	Cómo evoluciona una orden hasta quedar cerrada
Estados de la cuenta	Estado	Cómo funcionan la bancarrota simulada y el reinicio
Actividades de la compra	Comportamiento (proceso)	Cómo fluye la operación central del sistema

3.1 Diagrama de clases

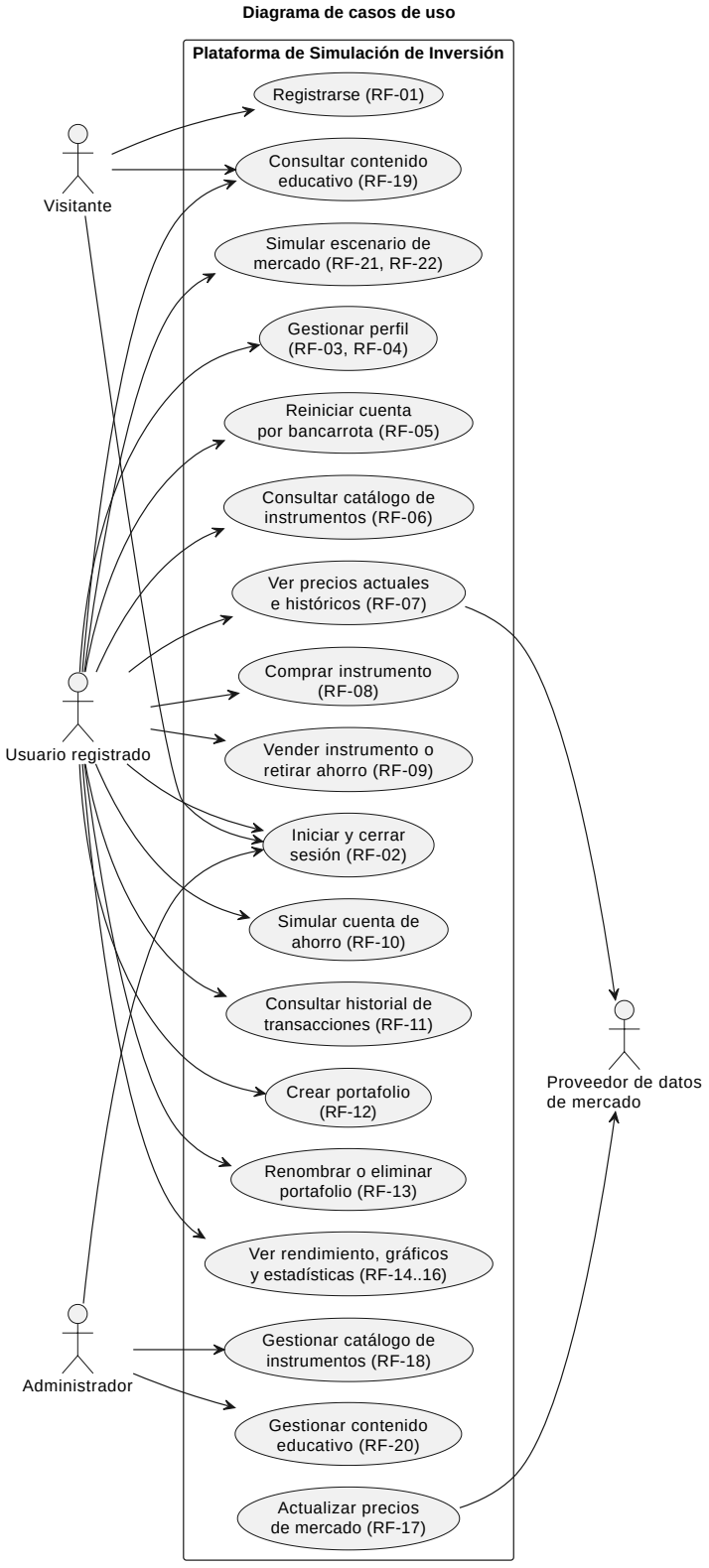


Decisiones de modelado:

- **Inversionista ◆ Portafolio** es una composición con multiplicidad 0..10, porque RN-05 limita a diez portafolios por usuario y un portafolio no existe sin su dueño.
- **Portafolio ◆ Posición** y **Portafolio ◆ DepósitoAhorro** son composiciones: al eliminar el portafolio se liquida su contenido (RN-07).

- **Inversionista ◇ Transacción** es una agregación, porque el historial sobrevive al reinicio por bancarrota y a la eliminación de un portafolio. Por eso Transacción se asocia con Portafolio con multiplicidad 0..1.
- **InstrumentoFinanciero** es abstracta y concentra símbolo, precio y fecha de actualización. Sus cuatro subclases solo añaden lo que las diferencia. Solo Criptomoneda es fraccionable (RN-02).
- **Cambios v1.1:**
  - `Transacción.tipo` incluye Depósito y Retiro.
  - Se agrega **DepósitoAhorro**, que guarda los parámetros que elige el usuario en RF-10 (monto, tasa, capitalización, plazo y fecha de inicio).
- **Visitante** no es una clase: es un rol sin estado persistente y solo aparece como actor.

3.2 Diagrama de casos de uso



Actor	Tipo	Responsabilidad
Visitante	Primario	Consulta el contenido educativo sin registrarse, se registra e inicia sesión.
Usuario registrado	Primario	Opera la simulación: portafolios, compras y ventas, ahorro, gráficos, escenarios y reinicio por bancarrota.
Administrador	Primario	Mantiene el catálogo de instrumentos (RF-18) y el contenido educativo (RF-20).

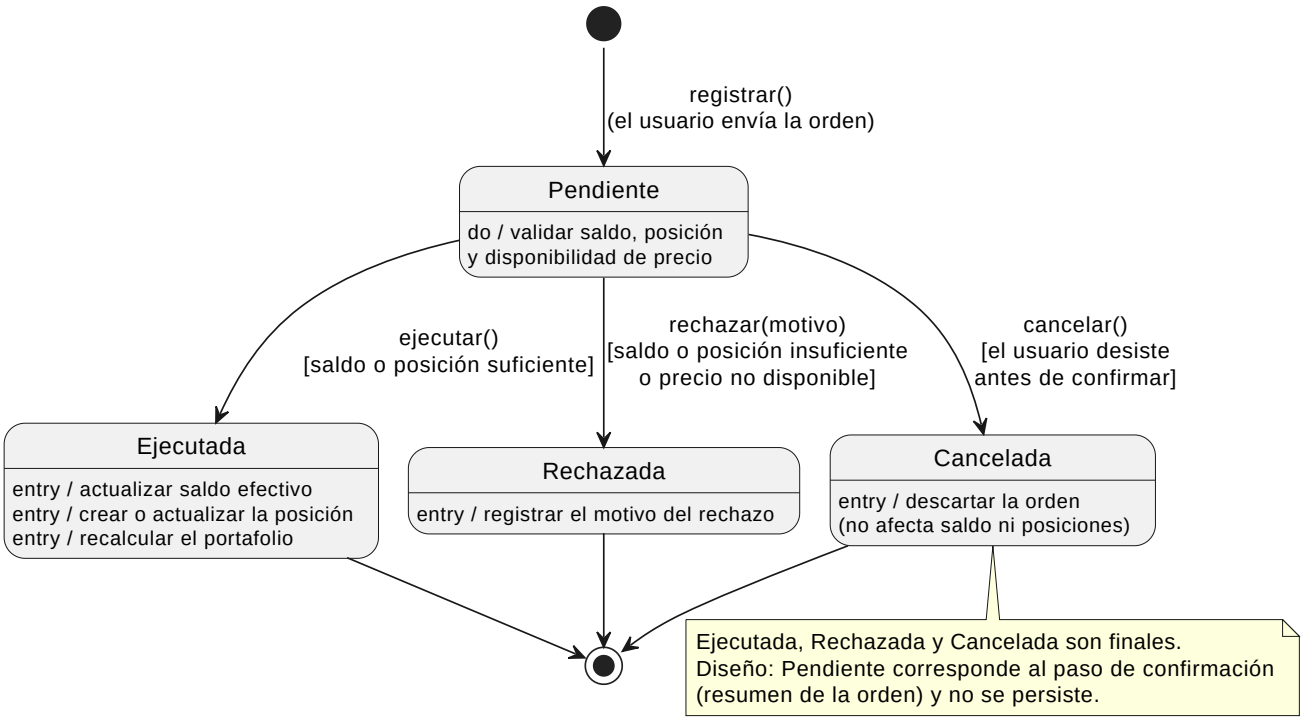
Actor	Tipo	Responsabilidad
Proveedor de datos de mercado	Secundario (externo)	Entrega los precios actuales y los históricos (RF-07, RF-17).

Cambios v1.1:

- Se agregan los casos *Simular cuenta de ahorro (RF-10)* y *Actualizar precios de mercado (RF-17)*.
- Los casos de uso del administrador citan sus RF.
- Cada caso de uso indica el requerimiento exacto del que proviene, en lugar del grupo de requerimientos.

3.3 Diagrama de estados de la Transacción

Diagrama de estados — objeto Transacción (RF-08, RF-09)



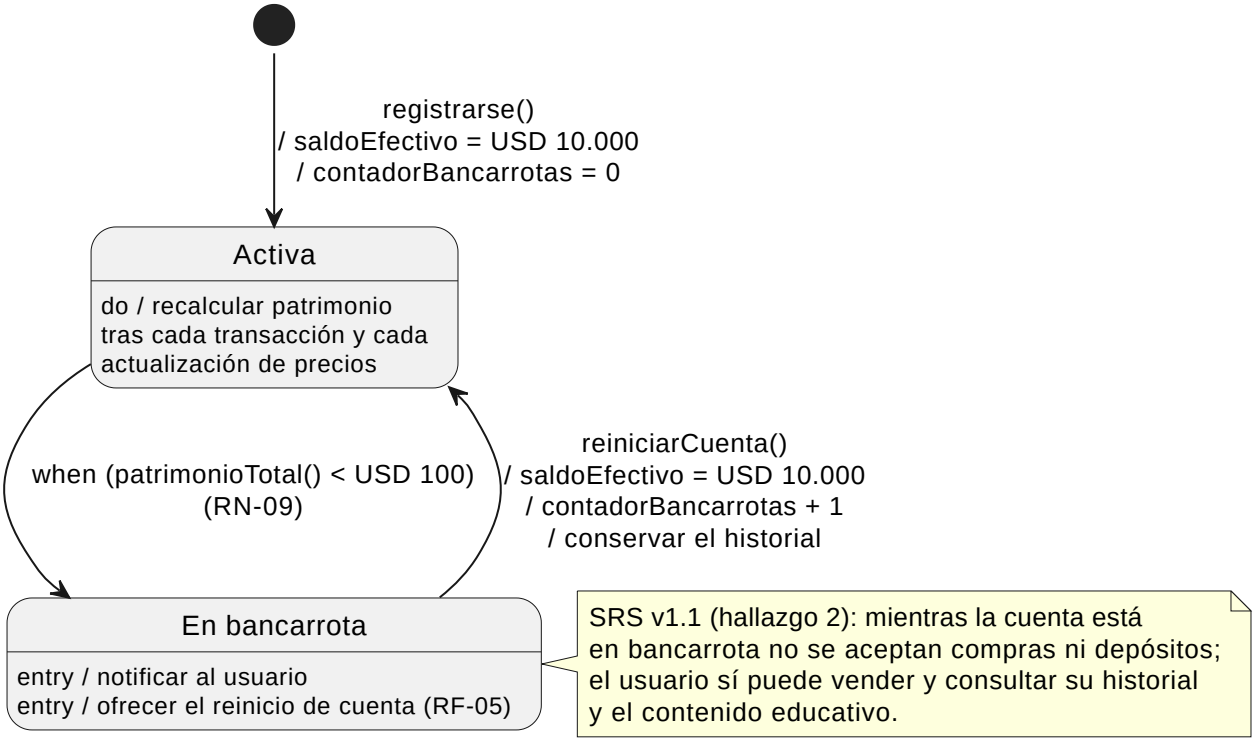
Estado origen	Evento	Guarda	Estado destino
(inicio)	registrar()	El usuario envía la orden	Pendiente
Pendiente	ejecutar()	Saldo o posición suficiente y precio disponible	Ejecutada
Pendiente	rechazar(motivo)	Saldo o posición insuficiente, o precio no disponible	Rechazada
Pendiente	cancelar()	El usuario desiste antes de confirmar	Cancelada

Los tres desenlaces son finales: una vez cerrada, la transacción queda en el historial y no vuelve a cambiar. Las acciones de entrada reflejan las reglas de negocio:

- **Ejecutada** mueve el saldo y la posición.
- **Rechazada** registra el motivo.
- **Cancelada** no toca nada.

3.4 Diagrama de estados de la cuenta del Inversionista

Diagrama de estados — cuenta del Inversionista (RF-05, RN-09)



- La transición a **En bancarrota** es un evento de cambio: ocurre cuando el patrimonio total cae por debajo de USD 100 (RN-09), sin que el usuario haga nada.
- El reinicio:
  - repone USD 10.000;
  - incrementa el contador;
  - **conserva el historial**, lo que justifica modelarlo como agregación.
- El supuesto del Taller sobre las órdenes en bancarrota quedó confirmado en el SRS v1.1.

3.5 Diagrama de actividades de la orden de compra



El flujo concentra las reglas de negocio más importantes:

- validación de decimales (RN-02);
- cálculo al último precio sin comisiones (RN-03);
- costo promedio ponderado (RN-06);
- los tres desenlaces posibles de una orden.

El par *fork/join* indica que el débito del saldo y la actualización de la posición deben completarse ambos antes de dar la orden por ejecutada. En el diseño esto se garantiza con una **transacción de base de datos atómica** (RNF-10).

La venta sigue el mismo flujo: cambia "saldo suficiente" por "posición suficiente" (RN-04) y el débito por un abono.

3.6 Trazabilidad con el SRS

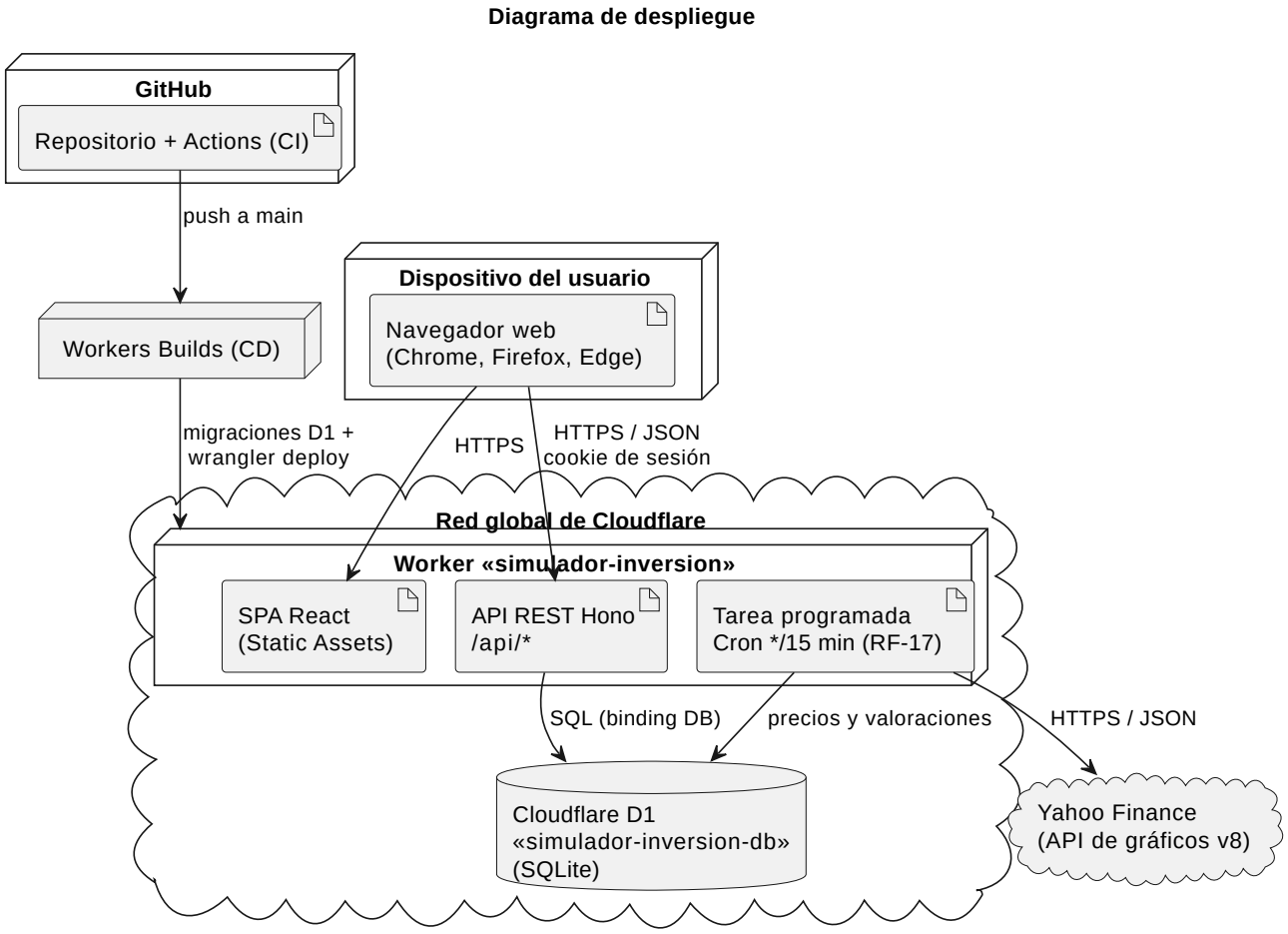
Requerimientos	Contenido	Diagramas
RF-01 a RF-05	Registro, perfil y reinicio por bancarrota	Clases, casos de uso, estados de la cuenta
RF-06 a RF-11	Catálogo, precios, compra, venta, ahorro e historial	Clases, casos de uso, estados de la transacción, actividades

Requerimientos	Contenido	Diagramas
RF-12 a RF-14	Portafolios	Clases, casos de uso
RF-15 a RF-18	Gráficos, estadísticas, precios y catálogo	Clases (CotizaciónHistórica, Proveedor), casos de uso
RF-19 a RF-20	Contenido educativo	Clases, casos de uso
RF-21 a RF-22	Escenarios	Clases (Escenario, FactorImpacto), casos de uso
RN-02, RN-03, RN-06	Cantidades, precio de ejecución y costo promedio	Actividades
RN-09	Bancarrota simulada	Clases, estados de la cuenta

4. Diseño del sistema

El diseño refina el modelo de análisis (sección 3) para una plataforma concreta: **Cloudflare Workers** (cómputo en el borde de la red) y **Cloudflare D1** (base de datos relacional SQLite). Las fuentes de los diagramas están en `docs/uml/diseno/`.

4.1 Arquitectura general y despliegue

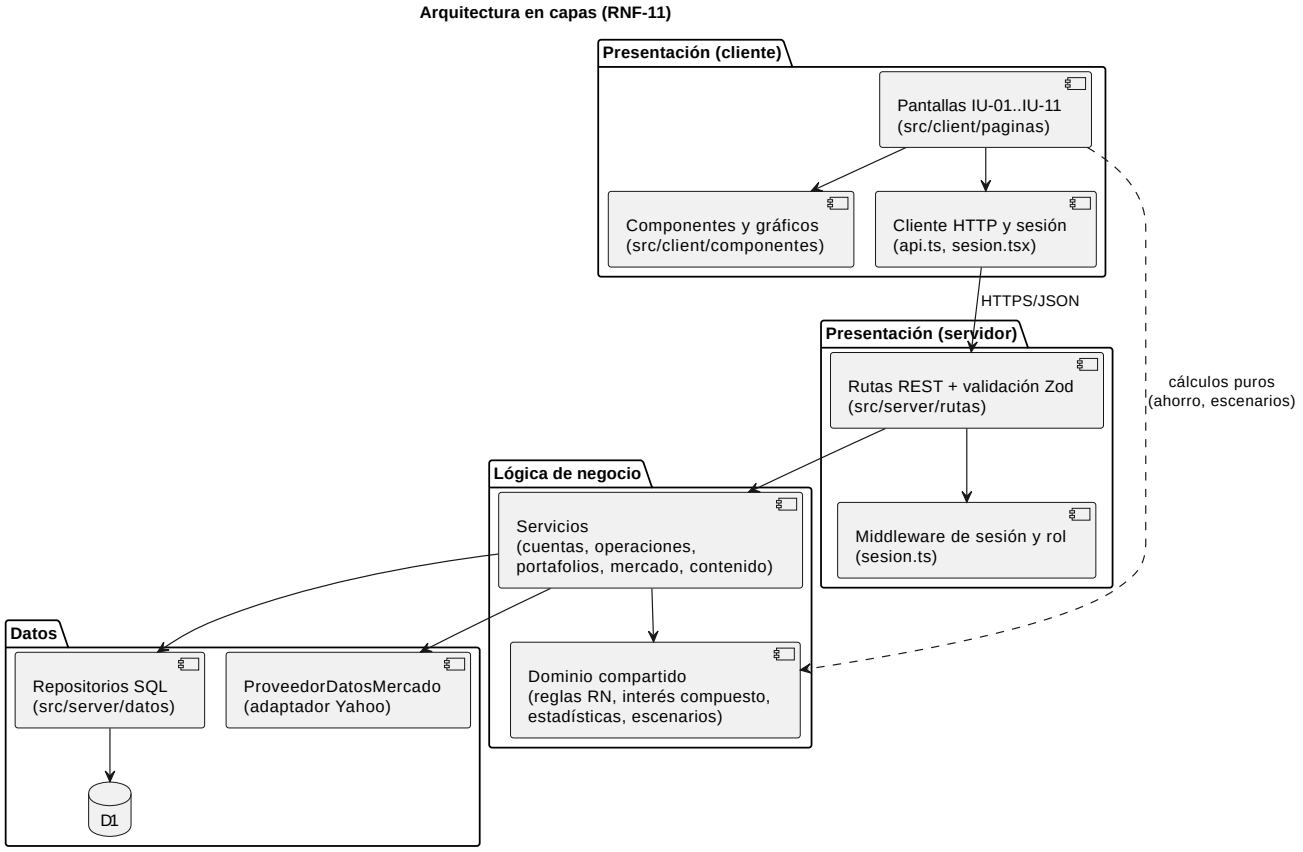


La arquitectura es **cliente-servidor**, como pide el SRS §2.1:

- **Cliente.** Aplicación de una sola página (SPA) en React que corre en el navegador. Se distribuye como recursos estáticos desde la red de Cloudflare.
- **Servidor.** Un único Worker atiende la API REST bajo `/api/*`, con JSON sobre HTTPS. También ejecuta la tarea programada de actualización de precios (RF-17) cada 15 minutos.
- **Datos.** Cloudflare D1, una base relacional administrada, se accede con un *binding*, sin credenciales en el código.
- **Proveedor externo.** Yahoo Finance entrega los precios actuales y el histórico (TBD-1).
- **Integración y entrega continuas:**
  - GitHub Actions ejecuta lint, tipos, pruebas y build en cada push y en cada PR.
  - Cloudflare Workers Builds despliega automáticamente cada integración a `main`: primero aplica las migraciones de D1 y luego publica el Worker.

URL de producción: https://simulador-inversion.juandanieltorresr.workers.dev.

4.2 Arquitectura en capas (RNF-11)



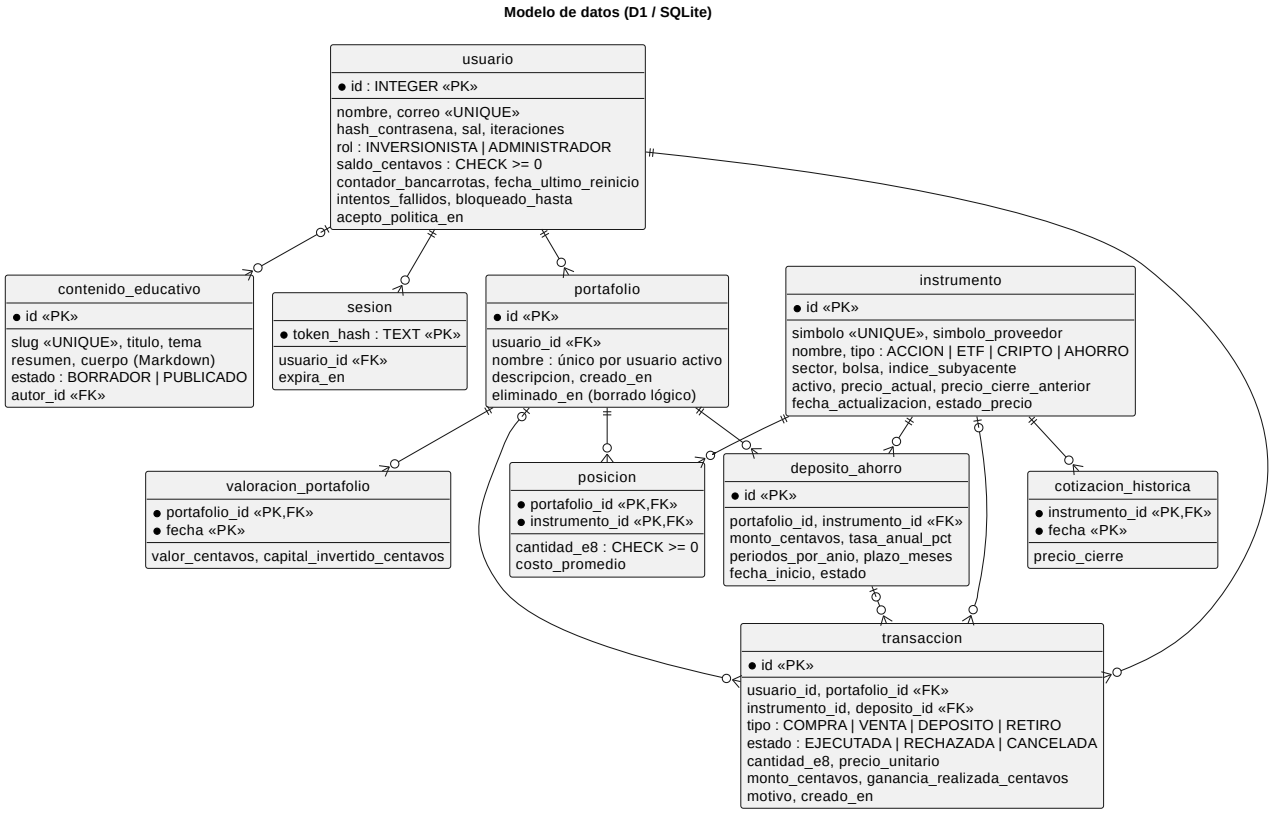
Capa	Ubicación	Responsabilidad
Presentación (cliente)	`src/client`	Pantallas IU-01 a IU-11, componentes, gráficos (Recharts) y cliente HTTP.
Presentación (servidor)	`src/server/rutas`, `src/server/sesion.ts`, `src/server/validacion.ts`	Contrato REST, validación de entradas (Zod), autenticación y autorización.
Lógica de negocio	`src/server/servicios`, `src/dominio`	Orquestación de operaciones y reglas de negocio. El dominio son funciones puras compartidas por cliente y servidor.
Datos	`src/server/datos`, `migrations/`	Repositorios SQL sobre D1 y el adaptador del proveedor de mercado.

Dominio compartido. El cliente importa la capa de dominio para dos cosas:

- validar al instante (cantidad, contraseña);
- calcular las proyecciones puras: ahorro (RF-10) y escenarios (RF-21, RF-22).

El servidor vuelve a validar todo lo que modifica datos. Así una sola implementación, probada al 100 %, sirve a ambos lados. Además se garantiza RN-08: el simulador de escenarios no tiene ningún camino para escribir en la base de datos.

4.3 Modelo de datos



Correspondencia con el modelo de análisis

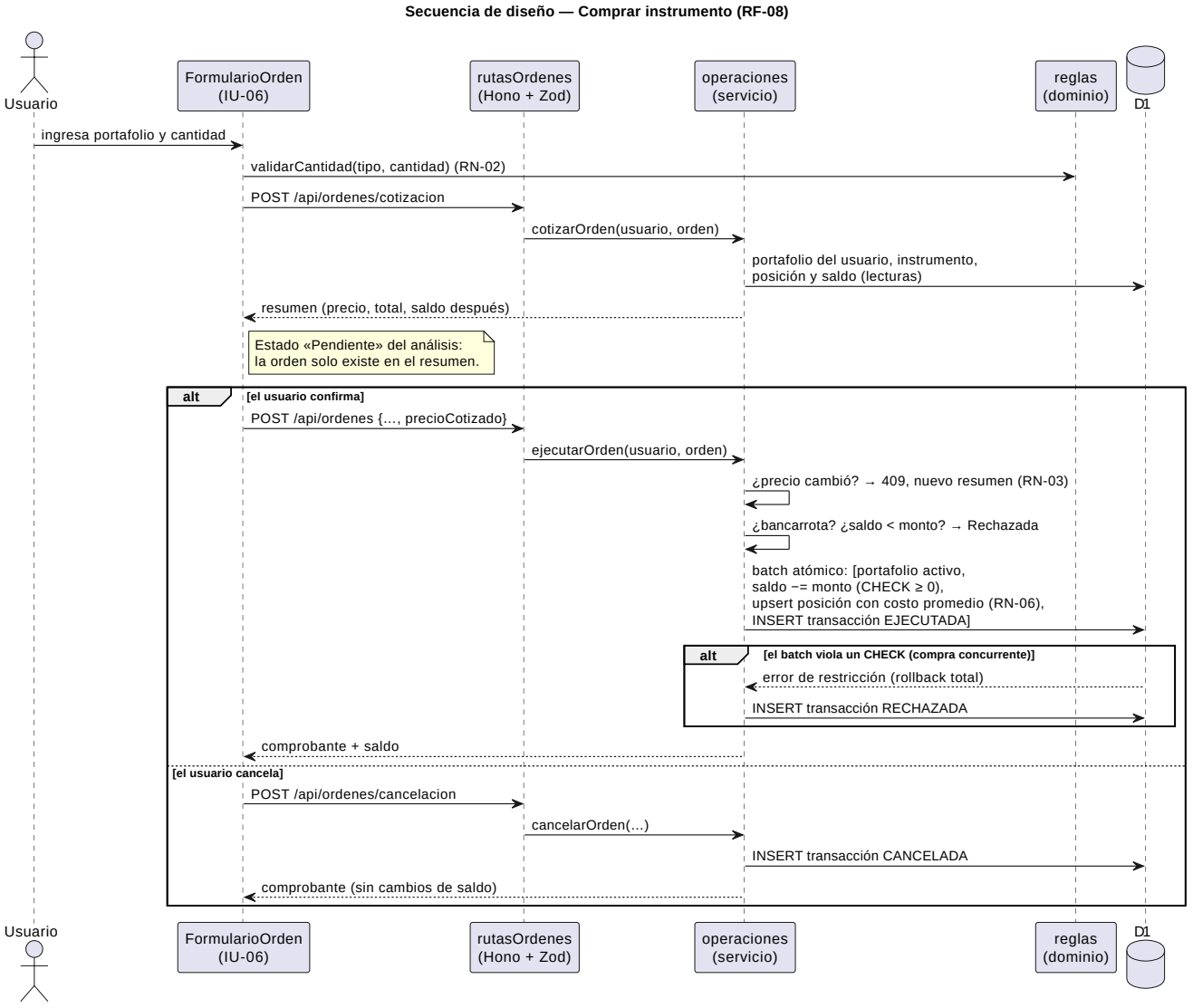
Clase de análisis	Tabla	Notas
Usuario, Inversionista, Administrador	`usuario`	Herencia en una sola tabla con `rol`.
Portafolio	`portafolio`	Borrado lógico, para que el historial sobreviva a la eliminación (agregación del análisis).
Posición	`posicion`	PK compuesta (portafolio, instrumento).
DepósitoAhorro	`deposito_ahorro`	Clase agregada en el SRS v1.1.
Transacción	`transaccion`	El estado Pendiente no se persiste (ver §4.7, D4).
InstrumentoFinanciero y subclases	`instrumento`	Herencia en una sola tabla con `tipo` y columnas opcionales (sector, bolsa, índice).
CotizaciónHistórica	`cotizacion_historica`	Un cierre diario por instrumento.
—	`valoracion_portafolio`	Valor diario de cada portafolio (RF-17), para el gráfico de RF-15.
ContenidoEducativo	`contenido_educativo`	Cuerpo en Markdown.
Escenario y FactorImpacto	—	Son constantes del dominio (`ESCENARIOS_PREDEFINIDOS`); no necesitan persistencia.

Invariantes en la base de datos (RNF-10)

- `CHECK (saldo_centavos >= 0)`: el saldo nunca puede ser negativo.
- `CHECK (cantidad_e8 >= 0)`: no se puede vender más de lo que se posee (RN-04).
- Índice único parcial `(usuario_id, nombre)` sobre los portafolios activos (RN-05).

- Disparador `trg_limite_portafolios`: máximo 10 portafolios activos (RN-05), incluso ante solicitudes simultáneas.

4.4 Comportamiento: secuencia de la compra



La orden de compra sigue el diagrama de actividades del análisis:

1. **Resumen de confirmación.** La cotización no tiene estado y equivale al estado *Pendiente*.
2. **Ejecución.** Ocurre dentro de un **único** `DB.batch()`, que D1 ejecuta como una transacción.
3. **Ante una compra simultánea**, si el saldo ya no alcanza, el `CHECK` aborta todo el batch. La orden se registra como **Rechazada**, sin cambios parciales.

La venta, el depósito y el retiro siguen el mismo patrón.

4.5 API REST

Todas las rutas usan el prefijo `/api`. Las marcadas con 🔒 requieren sesión; las marcadas con 🛡️ requieren el rol Administrador.

Método y ruta	Descripción	RF
`POST /auth/registro`	Registro con consentimiento; inicia la sesión	RF-01
`POST /auth/ingreso`, `POST /auth/salida`	Inicio y cierre de sesión	RF-02
`GET /auth/sesion`	Usuario de la sesión actual, o `null`	RF-02
🔒 `GET/PATCH /perfil`, `POST /perfil/reinicio`	Perfil, actualización y reinicio por bancarrota	RF-03, RF-04, RF-05
🔒 `GET /instrumentos?q=&tipo=`, `GET /instrumentos/:simbolo`	Catálogo y detalle	RF-06
🔒 `GET /instrumentos/:simbolo/historico?rango=`	Serie de cierres y estadísticas	RF-07, RF-16
🔒 `GET/POST /portafolios`, `GET/PATCH/DELETE /portafolios/:id`	Gestión de portafolios y liquidación	RF-12, RF-13, RF-14
🔒 `GET /portafolios/:id/valoracion?rango=`	Evolución del valor, estadísticas y composición	RF-15, RF-16
🔒 `POST /ordenes/cotizacion`, `POST /ordenes`, `POST /ordenes/cancelacion`	Resumen, ejecución y cancelación de órdenes	RF-08, RF-09
🔒 `POST /ahorro/depositos`, `POST /ahorro/depositos/:id/retiro`	Depósito y retiro de ahorro	RF-10, RF-09
🔒 `GET /transacciones?portafolioId=&desde=&hasta=&tipo=&simbolo=`	Historial con filtros	RF-11
`GET /contenido?tema=&q=`, `GET /contenido/:slug`	Contenido publicado, sin sesión	RF-19
🛡️ `GET/POST/PATCH /admin/instrumentos`	Catálogo	RF-18
🛡️ `GET/POST/PUT/DELETE /admin/contenido`	Contenido educativo	RF-20
`GET /salud`	Estado del servicio y de la base de datos (monitoreo de RNF-09)	—

Los errores siguen el formato `{ "error": "mensaje en español", "detalles": { "campo": "…" } }` con su código HTTP:

Código	Significado
400	Validación
401	Sin sesión
403	Sin rol
404	No existe o no es del usuario
409	Conflicto de negocio
422	Símbolo desconocido
423	Cuenta bloqueada

4.6 Tabla de pantallas

IU	Pantalla	Ruta	RF
—	Inicio (pública)	`/`	—

IU	Pantalla	Ruta	RF
IU-01	Registro e inicio de sesión	`/registro`, `/ingresar`	RF-01, RF-02
IU-02	Perfil de usuario	`/perfil`	RF-03, RF-04, RF-05
IU-03	Panel principal	`/panel`	RF-03, RF-12
IU-04	Catálogo de instrumentos	`/instrumentos`	RF-06
IU-05	Detalle de instrumento	`/instrumentos/:simbolo`	RF-07, RF-15, RF-16
IU-06	Operación de compra o venta	Formulario en IU-05 e IU-08	RF-08, RF-09
IU-07	Simulador de cuenta de ahorro	`/ahorro`	RF-10
IU-08	Detalle de portafolio	`/portafolios/:id`	RF-09, RF-11, RF-13, RF-14, RF-15, RF-16
IU-09	Centro de aprendizaje	`/aprende`, `/aprende/:slug`	RF-19
IU-10	Simulador de escenarios	`/escenarios`	RF-21, RF-22
IU-11	Panel de administración	`/admin`	RF-18, RF-20
—	Política de tratamiento de datos	`/privacidad`	RNF-07

4.7 Decisiones de diseño

#	Decisión	Alternativas consideradas	Justificación
D1	**Cloudflare Workers + D1 + Workers Builds** (TBD-2)	Servidor Node en una máquina virtual con PostgreSQL; Vercel + Supabase	Plan gratuito suficiente, HTTPS y CDN incluidos, base relacional administrada, despliegue automático desde GitHub y un solo lenguaje (TypeScript) en todas las capas. Costo: límites del plan gratuito, como 10 ms de CPU por solicitud.
D2	**Dinero en centavos enteros y cantidades en unidades × 10^8**	Números de punto flotante; tipo DECIMAL	SQLite no tiene DECIMAL. Con enteros la aritmética es exacta, los CHECK son fiables y se cumplen los criterios de aceptación al centavo: 1.500,00; 3.000,00; 1.126,83.
D3	**Atomicidad con `DB.batch()` y restricciones CHECK**	Bloqueos en la aplicación; leer, validar y escribir	D1 serializa las escrituras y ejecuta el batch como una transacción. La base de datos es la última barrera: ante compras o ventas simultáneas, solo una procede (RNF-10, verificado con pruebas de concurrencia).
D4	**El estado Pendiente no se persiste**	Guardar la orden como Pendiente y confirmarla después	Evita órdenes huérfanas, dobles confirmaciones y tareas de expiración. El resumen (cotización) cumple el papel de Pendiente; si el precio cambia antes de confirmar, se pide revisar (RN-03).
D5	**Proveedor tras una interfaz, con históricos precargados** (TBD-1)	Twelve Data o Finnhub (requieren clave); CoinGecko solo para cripto	Yahoo entrega acciones, ETF y cripto con 5 años de historia, sin clave. Al ser una API no oficial, el riesgo se mitiga así: 25.419 cierres en la migración 0004, el último precio guardado y el aviso de precio desactualizado (RNF-04). Cambiar de proveedor solo requiere otro adaptador. Se verificó en producción que Yahoo responde desde la red de Cloudflare.
D6	**Sesiones opacas en cookie HttpOnly y PBKDF2-SHA256 con 100.000 iteraciones**	JWT; bcrypt	La sesión opaca permite cerrarla de verdad (RF-02) y aplicar la expiración por inactividad (RNF-06). En la base de datos solo se guarda el hash del token. PBKDF2 está disponible de forma nativa en WebCrypto. Se midió en producción el 4 de noviembre de 2026: 6 de 6 derivaciones con 100.000 iteraciones sin exceder el límite de CPU del plan gratuito.

4.8 Seguridad

- **RNF-05.** Las contraseñas cumplen la política (8 o más caracteres, con letra y número) y se guardan como hash PBKDF2-SHA256 con sal de 128 bits. Solo se usa HTTPS: `workers.dev` no acepta HTTP plano y la SPA envía la cabecera HSTS.
- **RNF-06.**
  - **Sesión.** Tokens aleatorios de 256 bits en una cookie `HttpOnly; Secure; SameSite=Lax` con expiración deslizante de 30 minutos. Al cambiar la contraseña se cierran las demás sesiones.
  - **Bloqueo.** 15 minutos tras 5 intentos fallidos, con incremento atómico del contador.
  - **Autorización.** Rol Administrador para `/admin/*`, y toda consulta filtra por `usuario_id` (no se puede ver ni operar lo ajeno).
- **RNF-07.** Solo se piden nombre y correo. El consentimiento es obligatorio y se guarda su fecha. La política es accesible desde el pie de todas las pantallas.
- **Otras medidas:**
  - consultas SQL siempre parametrizadas;
  - middleware CSRF y cabeceras de seguridad (CSP, X-Frame-Options, nosniff);
  - el contenido Markdown se muestra sin HTML crudo;
  - mensaje genérico ante credenciales incorrectas y tiempo de respuesta igual cuando el correo no existe, para no permitir enumerar cuentas.

5. Validación del sistema

5.1 Estrategia de pruebas

Nivel	Herramienta	Alcance	Ubicación	Cantidad	Resultado
Unitarias	Vitest + cobertura v8	Capa de dominio: reglas de negocio y cálculos financieros	`tests/unit`	36	✅ 36/36
Integración	Vitest + `@cloudflare/vitest-plugin` (runtime de Workers + D1 real en Miniflare)	API completa: rutas, servicios, repositorios y base de datos	`tests/integracion`	43	✅ 43/43
Extremo a extremo	Playwright (Chromium) + axe-core	Flujos de usuario en el navegador, en escritorio y a 360 px	`tests/e2e`	5 × 2 vistas	✅ 10/10 en local y en producción
Carga	Script propio en Node.js (`tests/carga/carga.mjs`)	50 usuarios virtuales concurrentes contra producción	`tests/carga`	1 corrida de 10 min	p95 = 738 ms ✅; errores 13 % ⚠️ (§5.4)
Revisión estática	ESLint, TypeScript estricto y un agente revisor independiente	Calidad, tipos, seguridad y conformidad con el SRS	—	—	✅ 9 hallazgos corregidos

La **integración continua** (GitHub Actions) ejecuta lint, tipos, pruebas unitarias y de integración, cobertura y build en cada push y en cada pull request. Antes de integrar a `main` se verifica que la CI esté en verde.

Criterios de aceptación como pruebas. Cada criterio de aceptación numérico del SRS es una aserción automatizada con el mismo valor. Los valores esperados de las estadísticas (RF-16) se calcularon **de forma independiente** con el módulo `statistics` de Python sobre 25 cierres reales de AAPL, para cumplir la tolerancia de 0,01 que exige el SRS.

5.2 Cobertura

Cobertura de la capa de dominio (RNF-11 exige al menos 70 %):

Métrica	Resultado
Líneas	100 %
Sentencias	100 %
Funciones	100 %
Ramas	98,5 %

5.3 Matriz de trazabilidad: requerimiento → prueba → resultado

Requerimiento	Caso de prueba (criterio de aceptación verificado)	Tipo	Resultado
RF-01	CA1: con datos válidos se crea la cuenta con USD 10.000,00. CA2: no hay dos cuentas con el mismo correo (sin distinguir mayúsculas). Contraseña inválida, confirmación distinta y correo inválido son rechazados.	Integración	✅
RF-02	CA1 y CA2: inicio de sesión válido; tras cerrar sesión no hay acceso privado. Mensaje genérico para contraseña errada o cuenta inexistente. Bloqueo de 15 min tras 5 intentos, también con intentos simultáneos.	Integración	✅
RF-03	CA1: saldo = inicial − compras − depósitos + ventas + retiros; patrimonio = saldo + valor invertido.	Integración	✅
RF-04	CA1: los cambios se reflejan. CA2: se ingresa con la nueva contraseña y no con la anterior. La contraseña actual es obligatoria.	Integración	✅
RF-05 / RN-09	Con USD 80,00 se ofrece el reinicio; después hay USD 10.000,00, sin posiciones, contador +1 e historial conservado. Con patrimonio ≥ USD 100 no se permite reiniciar. Umbral exacto.	Unitaria e integración	✅
RF-06	CA1: búsqueda de "AAPL" y de "BTC". CA2: el filtro ETF solo devuelve ETF. Búsqueda sin resultados.	Integración	✅
RF-07	Serie del rango, aviso de rango incompleto y rango inválido.	Integración y E2E	✅
RF-08	CA1: 10 unidades a USD 150,00 cuestan USD 1.500,00. CA2: 0,05 BTC a USD 60.000,00 cuestan USD 3.000,00. CA3: las órdenes rechazadas o canceladas no modifican nada. Compra en 3 pasos y cancelación desde la interfaz.	Unitaria, integración y E2E	✅
RF-09	CA1: costo USD 150 y venta de 10 a USD 160 → abono de USD 1.600 y ganancia de USD 100. CA2: no se vende más de lo poseído, ni siquiera en paralelo.	Unitaria e integración	✅
RF-10	CA1: P = 1.000, r = 12 %, capitalización mensual, 1 año → USD 1.126,83. CA2: el depósito descuenta el saldo. Retiro con intereses, depósito mayor que el saldo y rangos de tasa y plazo.	Unitaria e integración	✅
RF-11	Orden del más reciente al más antiguo y filtros por tipo, instrumento y fechas (hora de Colombia).	Integración	✅
RF-12 / RN-05	Portafolio vacío en USD 0,00; nombre único sin distinguir mayúsculas; límite de 10.	Integración	✅
RF-13 / RN-07	Renombrar; eliminar liquida al precio actual, abona el saldo, conserva el historial y libera el nombre.	Integración	✅
RF-14	CA1: el valor total es la suma de las posiciones. CA2: el rendimiento coincide con el cálculo manual. Portafolio vacío.	Unitaria e integración	✅
RF-15	Serie de valoración con el valor actual y composición por tipo que suma 100 %.	Unitaria e integración	✅
RF-16	CA1: coincide con el cálculo independiente (tolerancia 0,01). "No disponible" con menos de 20 datos. Tendencia alcista y bajista.	Unitaria	✅
RF-17 / RNF-04	Actualiza precios y guarda la valoración diaria. Ante una falla del proveedor conserva el último precio y lo marca como desactualizado. Los instrumentos desactivados siguen recibiendo precio.	Integración	✅

Requerimiento	Caso de prueba (criterio de aceptación verificado)	Tipo	Resultado
RF-18	Símbolo desconocido (422) y repetido (409). Al desactivar, no se ofrece para compra pero sí se puede vender. Solo el administrador.	Integración	✅
RF-19	Lectura sin sesión; artículos en los 6 temas; búsqueda por término.	Integración	✅
RF-20	CA1: un borrador no es visible y al publicarlo sí. Validación de campos obligatorios. CA2: solo el administrador.	Integración	✅
RF-21 / RN-08	CA1: Crisis financiera sobre USD 1.000 solo en acciones → USD 700,00. Depósitos con la nueva tasa sin bajar de 0 %. Escenario alcista y portafolio mixto.	Unitaria	✅
RF-22	CA1: proyectado USD 1.000 con volatilidad del 10 % → rango USD 900–1.100. Validación de rangos.	Unitaria	✅
RN-02	Unidades enteras en acciones y ETF; hasta 8 decimales en cripto; cantidades inválidas rechazadas sin registrar la orden.	Unitaria e integración	✅
RN-03	Ejecución al último precio sin comisiones; si el precio cambia entre el resumen y la confirmación se pide revisar.	Integración	✅
RN-06	Costo promedio ponderado de dos compras (10 a 150 y 5 a 180 → 160).	Unitaria e integración	✅
RNF-01	Prueba de carga con 50 usuarios concurrentes en producción: p95 = 738 ms (límite: 3 s).	Carga	✅
RNF-02	La misma prueba (10 min, 50 usuarios). Saldos consistentes al final: 0 inconsistencias en 14.633 operaciones. Tasa de error: 13 % por la cuota diaria de escrituras del plan gratuito de D1 (§5.4).	Carga + SQL	⚠️ integridad ✅; repetir la medición de errores con presupuesto de escrituras
RNF-03	El aviso educativo y el enlace a la política de datos son visibles en 13 rutas; ningún formulario pide datos de pago.	E2E	✅
RNF-05	Hash con sal (no hay contraseñas legibles en la base de datos) y política de contraseña. Solo HTTPS (`workers.dev` + HSTS).	Integración e inspección	✅
RNF-06	Pantallas privadas redirigen al inicio de sesión. 401 sin sesión. 403 sin rol. 404 ante un portafolio ajeno. Expiración a los 30 min. Al cambiar la contraseña se cierran las demás sesiones.	Integración y E2E	✅
RNF-07	El registro no es posible sin aceptar la política; la política está en el pie de página y se guarda la fecha del consentimiento.	Integración y E2E	✅
RNF-08	Un usuario nuevo se registra y hace su primera compra en 3 pasos: 2,6 s automatizado (límite: 5 min).	E2E	✅ automatizada; ⏳ con personas (§5.6)
RNF-09	Disponibilidad ≥ 95 %.	Monitoreo	⏳ pendiente (§5.6)
RNF-10	Dos compras simultáneas que superan el saldo: solo una se ejecuta. Ventas simultáneas. El saldo nunca es negativo (CHECK).	Integración y carga	✅
RNF-11	Capas en el repositorio y cobertura del dominio de 100 % en líneas.	Inspección y cobertura	✅
RNF-12	Sin desborde horizontal a 360 px; 0 violaciones WCAG 2.1 A/AA (axe) en 13 rutas (todas las IU excepto IU-11, que requiere cuenta de administrador), en escritorio y en móvil.	E2E	✅ en Chromium; ⏳ Firefox y Edge (§5.6)

5.4 Prueba de carga (RNF-01, RNF-02)

Configuración (corrida 1, 04/11/2026, 02:04–02:14 UTC):

- **Destino:** producción.

- **Usuarios:** 50 usuarios virtuales concurrentes, cada uno con su propia cuenta y sesión.
- **Ciclo de cada usuario:** perfil → catálogo → histórico de 1 mes → compra → detalle del portafolio → venta, con una pausa de 1 s. Son unas 2,4 solicitudes por segundo por usuario, alrededor de 10 veces el ritmo de una persona: es una prueba de **estrés**.

Métrica	Resultado
Solicitudes	58.234 en 602 s (96,8 solicitudes/s)
Tiempo de respuesta	p50 293 ms · **p95 738 ms** · p99 816 ms · máximo 3.086 ms
Errores	7.614 (13,07 %), todos HTTP 500
Consistencia final (SQL)	56 usuarios de prueba, 14.633 transacciones ejecutadas: **0 saldos que no cumplan la fórmula de RF-03** y 0 saldos negativos

Ruta	Solicitudes	p50 (ms)	p95 (ms)
`GET /perfil`	9.689	230	334
`GET /instrumentos`	9.689	198	280
`GET /instrumentos/:s/historico`	9.689	281	363
`POST /ordenes` (compra)	9.689	664	804
`GET /portafolios/:id`	9.689	364	455
`POST /ordenes` (venta)	9.689	512	782

Análisis de causa. La primera hipótesis era la saturación de la cola de D1, que atiende una consulta a la vez. **Se descartó con evidencia.** Al intentar limpiar los datos de prueba, D1 respondió con el error `7500: Your account has exceeded D1's free tier daily row write limit`. La causa real fue esta:

- El plan gratuito de D1 permite **100.000 filas escritas por día**.
- La prueba ejecutó unas 14.600 operaciones de unas 7 filas cada una (saldo, posición y transacción, más sus índices), además de los 25.419 cierres históricos precargados ese día.
- Así agotó la cuota durante la ejecución. A partir de ese momento fallaron todas las escrituras, incluida la renovación de la sesión, y por eso también fallaron algunas consultas de lectura.

Conclusiones de la prueba:

- **RNF-01 se cumple** con amplio margen: p95 de 738 ms bajo una carga 10 veces mayor que la de usuarios reales.
- **La integridad de RNF-02 y RNF-10 se cumple**: aun con escrituras fallando, ninguna operación quedó a medias y todos los saldos son consistentes. La atomicidad por lotes y las restricciones CHECK funcionaron como se diseñaron.
- **La tasa de error de RNF-02 (< 1 %) no se cumplió en esta corrida**, por una restricción de la plataforma y no de la lógica del sistema.

Acciones correctivas (aplicadas):

1. Lecturas de cada endpoint agrupadas en un solo `DB.batch()`, con lo que las idas a D1 bajan a la mitad.
2. Renovación de sesión de *mejor esfuerzo*: si la cuota se agota, las consultas siguen funcionando.
3. Respuesta 503 con un mensaje claro cuando la cuota se agota.
4. Escrituras del cron reducidas en cerca de 80 %: cierre del día por ciclo y valoración una vez al día.

Pendiente: repetir la medición de la tasa de error con un perfil realista y dentro del presupuesto diario de escrituras, después del reinicio de la cuota a medianoche UTC:

```
npm run test:carga -- https://simulador-inversion.juandanieltorresr.workers.dev
50 600 10000
```

Esa corrida usa 50 usuarios con pausas de 10 s durante 10 minutos, unas 35.000 filas escritas.

5.5 Defectos encontrados por las pruebas y las revisiones

Origen	Defecto	Corrección
Pruebas de integración	Las ventas siempre se rechazaban: SQLite evalúa el `CHECK` sobre la fila que intenta insertar antes de resolver el `UPSERT`.	Nuevo patrón: `INSERT OR IGNORE` de la posición seguido de un `UPDATE` que resta.
Revisión independiente (IA)	Nueve hallazgos de concurrencia, seguridad y fechas (detalle en el commit `7fd5193`).	Todos corregidos, con 5 pruebas de regresión.
E2E (axe)	Las tablas con desplazamiento no eran accesibles por teclado (`scrollable-region-focusable`).	Las regiones ahora son enfocables y tienen etiqueta.
E2E (360 px)	El campo de búsqueda desbordaba horizontalmente.	`min-width: 0` en el campo flexible.
Recorrido manual (Playwright)	La pantalla quedaba en blanco: `window.scrollTo` devuelve una promesa en Chromium y React la trataba como función de limpieza.	El efecto ya no devuelve ese valor.

5.6 Pruebas pendientes a cargo del equipo

Estas pruebas requieren personas o tiempo real, así que **no las ejecutó el asistente de IA**. Quedan con su protocolo:

1. **Usabilidad (RNF-08).** Prueba con 5 personas que no conozcan la plataforma.
    - Consigna: "regístrate y compra tu primera acción", sin ayuda.
    - Registrar por persona el tiempo (cronómetro), el número de errores y los comentarios.
    - Criterio: al menos 4 de 5 terminan en 5 minutos o menos.
    - Plantilla de registro:

Persona	Tiempo	¿Terminó sin ayuda?	Errores y observaciones
1			

2. **Disponibilidad (RNF-09).**
    - Crear en UptimeRobot (gratuito) un monitor HTTP cada 5 minutos a `https://simulador-inversion.juandanieltorresr.workers.dev/api/salud`.
    - Tras 2 semanas, adjuntar la captura del porcentaje de disponibilidad. Criterio: 95 % o más.
3. **Navegadores (RNF-12).** Recorrer las funciones principales en las dos últimas versiones de Firefox y de Edge: registro, compra, portafolio, ahorro, escenarios y aprendizaje.

6. Descripción del proceso

> ⚠️ **Por confirmar por el equipo.** Esta sección reconstruye el proceso a partir de la evidencia disponible: el historial de revisiones del SRS, la fecha del Taller UML y el historial de commits. La **distribución de roles y de trabajo** de la §6.3 es una *propuesta* que el equipo debe ajustar a lo que realmente hizo cada integrante antes de entregar.

6.1 Modelo de proceso

Se usó un proceso **ágil basado en Scrum**, como fija el SRS §2.5, con estos elementos:

- **Product backlog** formado por los requerimientos del SRS (RU → RF/RNF), priorizados con la prioridad de desarrollo del propio SRS: primero los Alta, luego los Media y por último los Baja.
- **Sprints con entregable verificable.** Cada sprint termina con un incremento revisable: un documento aprobado o software desplegado.
- **Definición de terminado (DoD) del producto:**
  - código en el repositorio;
  - lint y tipos sin errores;
  - pruebas en verde en la CI;
  - requerimiento trazado a su prueba;
  - integración a `main` mediante pull request y despliegue automático.
- **Control de versiones.** Git y GitHub con ramas de trabajo, pull requests y commits en estilo convencional. Ver la guía de contribución.

6.2 Sprints

Sprint	Periodo	Objetivo	Incremento y evidencia
0	Semanas 1–2	Propuesta y conformación del equipo	Selección del proyecto *Plataforma de Simulación de Inversión* y propuesta (entrega de la semana 2).
1	Semanas 3–5	Ingeniería de requerimientos	Necesidades y RU; prototipos del SRS en clase (v0.1 identificación inicial, v0.2 refinamiento).
2	Hasta el 25/09/2026	Especificación y análisis	**SRS v1.0** (18/09/2026): 22 RF, 12 RNF, 9 RN, 11 IU. **Taller de análisis UML** (25/09/2026): clases, casos de uso, estados y actividades.
3	04/11/2026	Construcción asistida por IA	Software desplegado en producción, informe 01–04 y pruebas. Las iteraciones de este sprint están en la tabla siguiente.
4	Hasta la semana 16	Validación con usuarios y entrega final	Pendiente: prueba de usabilidad con 5 usuarios (RNF-08), 2 semanas de monitoreo de disponibilidad (RNF-09), pruebas en Firefox y Edge (RNF-12), video demostrativo y presentación.

Iteraciones del sprint 3 (historial de commits, hora UTC)

Iteración	Commit	Resultado
3.0 Plan	—	Lectura completa del SRS y del Taller, plan aprobado por el equipo, decisiones (despliegue, proveedor de datos, formato del informe) y revisión crítica del plan por un agente independiente.

Iteración	Commit	Resultado
3.1 Esqueleto desplegable	`bfe735e`	Repositorio profesional, CI, D1 creada y primer despliegue. En producción se verificó el proveedor de datos y el costo de PBKDF2.
3.2 Dominio y datos	`0e364e8`	Reglas y cálculos financieros con 35 pruebas (en ese commit), contenido educativo y 25.419 cierres históricos.
3.3 Informe 01–03	`82d1482`	Problema, requerimientos (SRS v1.1) y diagramas de análisis corregidos.
3.4 API	`c9137e5`	RF-01 a RF-22, cron de RF-17 y 37 pruebas de integración (en ese commit; hoy son 43).
3.5 Revisión crítica	`7fd5193`	Un agente revisor independiente encontró 9 defectos; se corrigieron todos y se agregaron pruebas de regresión.
3.6 Interfaz	`01bbd6d`	Pantallas IU-01 a IU-11.
3.7 Validación E2E	`ff5a4d9`	Suite Playwright con axe: encontró y corrigió 2 fallas de accesibilidad y 1 desborde a 360 px.
3.8 Integración	PR #1	Integración a `main` y despliegue automático a producción.
3.9 Informe y cierre	commits posteriores	Diseño, validación, proceso, conclusiones, perspectivas, PDF y auditoría final.

6.3 Roles y distribución del trabajo (propuesta)

Rol Scrum	Integrante (propuesto)	Responsabilidades
Product Owner	Juan Daniel Torres Ramírez	Priorizar el backlog, validar los criterios de aceptación del SRS y tomar las decisiones de producto (proveedor, despliegue, alcance).
Scrum Master	Camilo Andrés Velandia Mendoza	Facilitar las reuniones, eliminar impedimentos y velar por la definición de terminado y el control de versiones.
Equipo de desarrollo	Los tres integrantes	Análisis, diseño, implementación, pruebas y documentación.

Área de trabajo	Responsable principal (propuesto)	Apoyo
Requerimientos (SRS) y validación con usuarios	Juan Daniel Torres Ramírez	Todo el equipo
Análisis UML (Taller) y diseño	Camilo Andrés Velandia Mendoza	Santiago Fonseca Hernández
Pruebas, accesibilidad e informe de validación	Santiago Fonseca Hernández	Camilo Andrés Velandia Mendoza
Supervisión del asistente de IA, revisión de código e integración	Juan Daniel Torres Ramírez	Todo el equipo

6.4 Uso de inteligencia artificial

Siguiendo la orientación del docente, la construcción (sprint 3) se hizo con **Claude Code**, el asistente de ingeniería de software de Anthropic, que trabajó sobre el repositorio con supervisión del equipo.

Lo que hizo la IA:

- Leyó de forma completa el SRS v1.0 y el Taller UML y propuso un plan, que el equipo aprobó.
- Generó el código, las migraciones, las pruebas, los diagramas de diseño y la mayor parte del informe técnico.
- Se **autorrevisó** con agentes independientes:

    - **Revisión del plan**, antes de escribir código. Cambió el orden del trabajo: primero un despliegue mínimo y verificar riesgos de la plataforma.
    - **Revisión de la API contra el SRS.** Encontró 9 defectos de concurrencia, seguridad y fechas, que se corrigieron con pruebas de regresión.
    - **Auditoría final de trazabilidad.**
- Detectó un error del propio Taller: el hallazgo 3 afirmaba que el administrador no tenía RF, pero RF-18 y RF-20 sí existen. También detectó vacíos del SRS que se documentaron en la versión 1.1.

Lo que hizo el equipo:

- Definió el problema, los requerimientos (SRS) y el análisis (Taller UML), que son la base de todo el trabajo.
- Tomó las decisiones con impacto: plataforma de despliegue, proveedor de datos, formato del informe y alcance.
- Conectó la infraestructura (Cloudflare Workers Builds) y revisa y valida el resultado.

Controles de calidad aplicados al trabajo generado:

- Cada criterio de aceptación numérico del SRS está automatizado como prueba.
- Las estadísticas se contrastaron con un cálculo independiente en Python.
- La CI ejecuta todas las pruebas en cada cambio.

Limitaciones declaradas:

- La IA no reemplaza las pruebas con personas: la usabilidad (RNF-08) y la disponibilidad prolongada (RNF-09) quedan a cargo del equipo.
- Toda decisión de diseño debe poder explicarla el equipo. Por eso cada una está justificada en la §4.7.

7. Conclusiones

7.1 Síntesis de resultados

Aspecto	Resultado
Requerimientos funcionales	**22 de 22 implementados** (RF-01 a RF-22), cada uno con al menos una prueba automatizada de sus criterios de aceptación.
Reglas de negocio	**9 de 9** implementadas; las críticas (RN-04, RN-05) se imponen también en la base de datos.
Interfaces de usuario	**11 de 11** pantallas del SRS, más el inicio, la política de datos y el detalle de artículo.
Requerimientos no funcionales	**8 de 12 verificados** (RNF-01, 03, 04, 05, 06, 07, 10 y 11). **3 parciales**: RNF-02 (integridad verificada bajo carga; falta repetir la medición de errores), RNF-08 (automatizado; falta la prueba con personas) y RNF-12 (solo Chromium). **1 pendiente**: RNF-09 (2 semanas de monitoreo). Ver §5.4 y §5.6.
Pruebas	36 unitarias, 43 de integración y 10 E2E, todas en verde. Cobertura del dominio: 100 % de líneas.
Despliegue	En producción, con integración y entrega continuas: https://simulador-inversion.juandanieltorresr.workers.dev.

7.2 Análisis

1. **Un buen SRS hizo posible construir rápido y verificar con precisión.** El equipo escribió criterios de aceptación con valores concretos: USD 1.500,00, USD 1.126,83, USD 700,00, el rango de USD 900 a 1.100. Así cada requerimiento se convirtió directamente en una prueba automatizada. Es la principal lección del curso puesta en práctica: un requerimiento verificable ahorra discusiones y defectos.

2. **El análisis UML guió el diseño, pero necesitó refinamiento.** El estado *Pendiente* que agregó el Taller resultó correcto como concepto, pero persistirlo creaba riesgos (órdenes huérfanas, dobles confirmaciones). En el diseño se materializó como el paso de confirmación. El análisis responde *qué* ocurre; el diseño, *cómo* se garantiza.

3. **La revisión crítica encontró errores en todas las etapas, también en la nuestra:**

   - un hallazgo del Taller era incorrecto (RF-18 y RF-20 sí existían);
   - el SRS tenía vacíos que se documentaron en su versión 1.1;
   - el revisor independiente encontró 9 defectos de concurrencia y seguridad en la API;
   - las pruebas automatizadas revelaron fallas de accesibilidad y un comportamiento no evidente de SQLite.

   Ninguno de estos defectos se habría visto sin pruebas ni revisiones.

4. **La integridad de los datos debe residir en la base de datos.** Las restricciones CHECK y la ejecución atómica por lotes garantizan que el saldo nunca sea negativo, aun con solicitudes simultáneas (RNF-10). Las validaciones de la aplicación mejoran la experiencia, pero la base de datos es la que da la garantía.

5. **Las cuotas de la plataforma también son requerimientos no funcionales.** La prueba de carga confirmó el rendimiento (p95 de 738 ms) y la integridad (0 saldos inconsistentes). También agotó la cuota diaria de escrituras del plan gratuito de D1, algo que el SRS no contemplaba. Una prueba de carga debe presupuestar los límites del proveedor, y el diseño debe degradarse con elegancia cuando se alcanzan (§5.4).

6. **La IA aceleró la construcción, pero no reemplazó el criterio del equipo.** El asistente produjo código, pruebas y documentación en una fracción del tiempo habitual. Aun así, el resultado depende de un SRS bien hecho y de las decisiones del equipo (plataforma, proveedor, alcance). Las verificaciones con personas reales siguen siendo responsabilidad humana.

7.3 Riesgos y limitaciones

- **Proveedor de datos.** Yahoo Finance es una API no oficial y puede cambiar o bloquear el acceso. La mitigación (último precio guardado e históricos precargados) mantiene la aplicación operativa, pero sin precios nuevos.
- **Plan gratuito de Cloudflare.** 100.000 solicitudes diarias y 10 ms de CPU por solicitud son suficientes para un curso, pero limitan una adopción masiva.
- **Historial de valor de portafolios.** La gráfica de valor de un portafolio comienza el día en que se crea, porque las valoraciones se registran diariamente desde entonces.

8. Perspectivas

Mejoras identificadas durante el proyecto, ordenadas por el valor que aportarían.

8.1 Validación pendiente (antes de la entrega final)

1. **Prueba de usabilidad con 5 usuarios nuevos (RNF-08).** Seguir el protocolo de la §5.6 y registrar tiempos y observaciones. La prueba automatizada indica que el flujo cabe con holgura en 5 minutos, pero eso no sustituye a personas reales.
2. **Monitoreo de disponibilidad durante 2 semanas (RNF-09)** con un servicio externo sobre `/api/salud`.
3. **Pruebas manuales en Firefox y Edge (RNF-12).** La suite automatizada solo se ejecutó en Chromium.

8.2 Producto

- **Historial de valor desde el primer día.** Hoy la curva de un portafolio empieza el día en que se crea, porque las valoraciones se guardan una vez al día. El valor de días pasados podría reconstruirse a partir de las transacciones y de los cierres históricos, con lo que se tendría una curva desde la primera compra. También permitiría un modo de *backtesting*: "¿qué habría pasado si hubiera comprado hace un año?".
- **Más tipos de orden y de eventos.** Órdenes límite y *stop-loss*, dividendos y *splits*. El SRS los excluye hoy (RN-03), pero son el siguiente paso natural para el aprendizaje.
- **Comparación con un índice de referencia** (por ejemplo, el S&P 500) en el rendimiento del portafolio, y ranking anónimo entre estudiantes de un mismo curso para fomentar el aprendizaje.
- **Contenido educativo interactivo**: cuestionarios por tema y recomendaciones según el comportamiento del usuario. Por ejemplo, sugerir el artículo de diversificación cuando más del 80 % del portafolio está en un solo tipo.
- **Aplicación instalable (PWA)** y notificaciones. Las aplicaciones nativas están fuera del alcance (SRS §1.4).

8.3 Técnica

- **Proveedor de datos oficial.** Yahoo Finance es una API no oficial. Para un uso sostenido conviene un proveedor con contrato y clave (Twelve Data, Alpha Vantage o Finnhub). La interfaz `ProveedorDatosMercado` permite cambiarlo sin tocar la lógica.
- **Observabilidad y alertas.** Alertas automáticas cuando el cron registre fallas consecutivas del proveedor, y métricas de latencia por ruta.
- **Pruebas de carga más largas y en CI**, y pruebas E2E en Firefox y WebKit ejecutadas en GitHub Actions.
- **Entornos separados**: una base D1 de *staging* para las vistas previas de ramas, de modo que las versiones de prueba no compartan la base de producción.
- **Recuperación de cuenta** por correo. El SRS §3.4 excluye el envío de correos, pero es la función que más extrañaría un usuario real.

8.4 Proceso

- Registrar el seguimiento del backlog (issues y tablero) desde el primer sprint, para que la evidencia del proceso no dependa solo de los commits.
- Incorporar la revisión crítica por agentes de IA como práctica fija de cada pull request, junto con la revisión humana.